VERILOG · HOMEWORK

0101 Sequence Detector and Button Debouncer

Mealy · Moore FSM / button_debouncer / FSM_RUN_STOP

정안성 · 2026.10.12

목차

00

개념 정리

01 **Debounce 개념**
FPGA 버튼 회로 · Bounce · 현업 해결 방법 · SIPO · Tick
p.3 – 11

02 **Sampling · Filter**
Impulse train · Spectrum 복사 · Aliasing · |H(f)| · AND
p.12 – 18

설계 · 검증

03 **Homework-1 설계**
Mealy · Moore State Diagram
p.19 – 20

04 **Homework-1 시뮬레이션**
환경 · Timing · 패턴 검출 · 패턴 없음 · next logic
p.21 – 29

05 **Homework-2 설계**
Switch → Button · Block Diagram · FSM · Debouncer
p.30 – 34

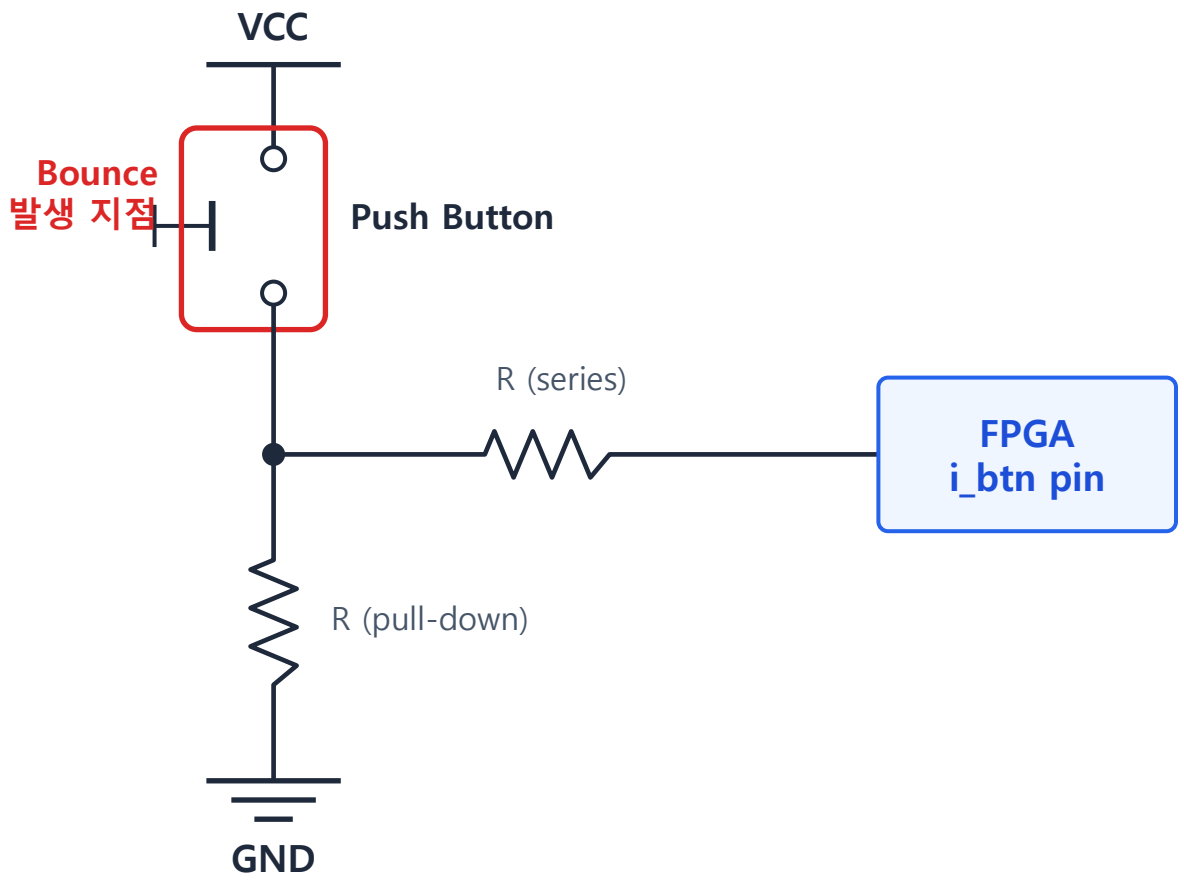
06 **Homework-2 시뮬레이션**
짧은 Pulse · Shift · 비동기 입력 · 최소 유지 시간
p.35 – 40

07 **결론**
설계 결과 · 고찰
p.41

Push Button Pulls the FPGA Pin High Through a Contact

01

Push Button Input Circuit



1. **Push** : 접점 연결 → pin = VCC → 1
2. **Pop** : 접점 분리 → pull-down R 경유 GND → 0
3. **Series R** : pin 유입 전류 제한 · 보호

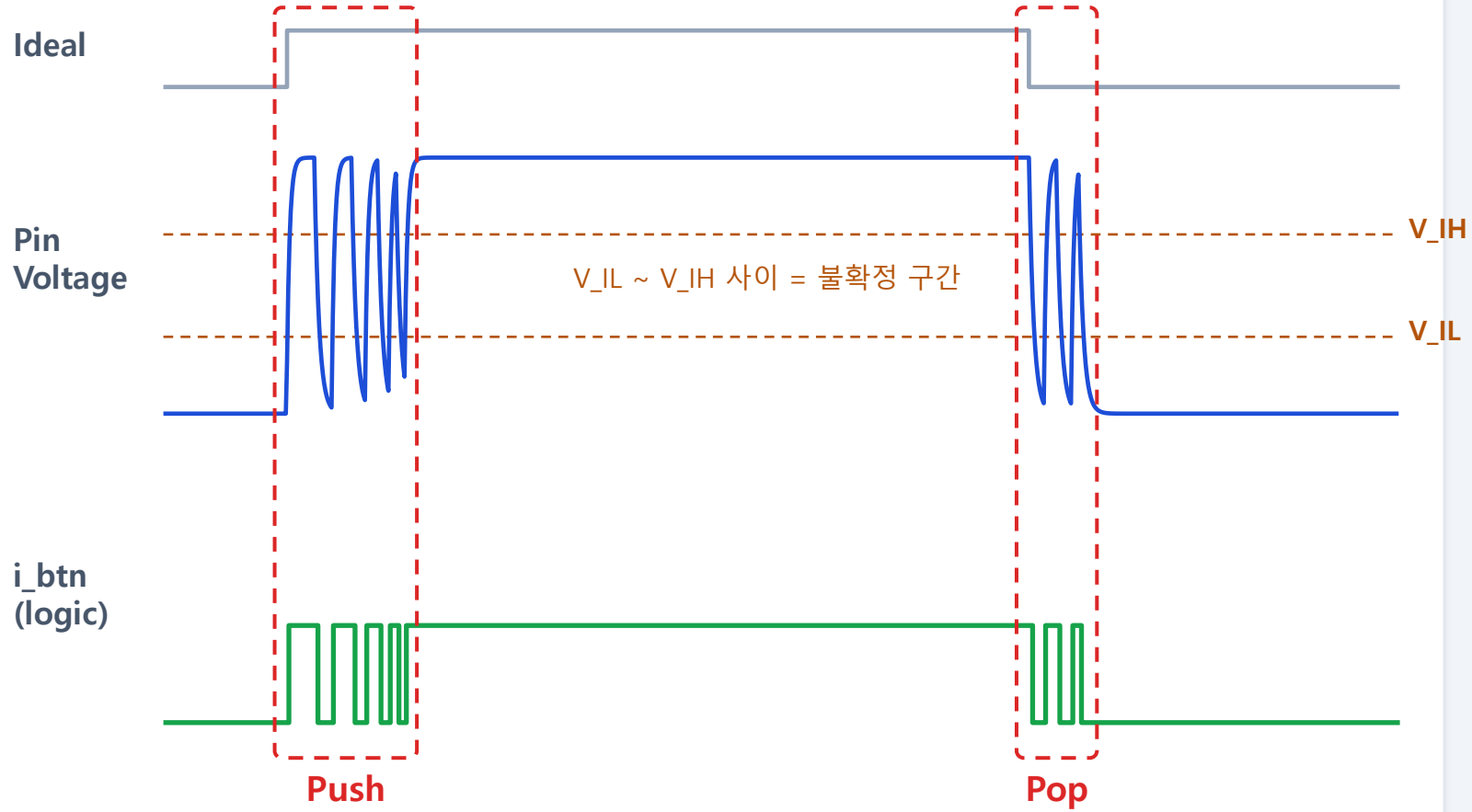
접점 = 금속 판의 기계적 충돌

- 충돌 순간 탄성 반발 → 붙었다 떨어짐 반복
- 분리 순간에도 진동 → 0 / 1 반복
- FPGA pin = 이 전압 변화를 그대로 수신

Contact Bounce Repeats 0 / 1 at Push and Pop

01

Pin Voltage → FPGA Input Level



정의

Mechanical scattering noise
금속 접점의 탄성 충돌로 한 번의 조작에 0 / 1 전이가 여러 번 발생하는 현상

발생 원인

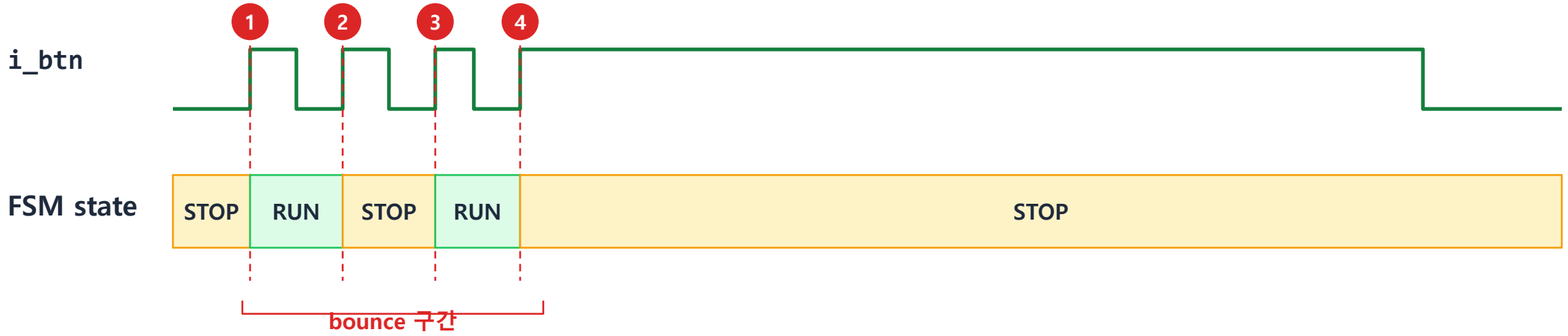
- Push: 접점 충돌 → 반발 → 재접촉
- Pop: 접점 분리 순간 진동
- 전압이 V_IL ~ V_IH를 여러 번 통과

특징

- Push · Pop 양쪽에서 발생
- pulse 폭 · 개수 불규칙
- push마다 다른 패턴

One Press Becomes Several Rising Edges and Toggles

i_btn 직결 시 FSM 동작



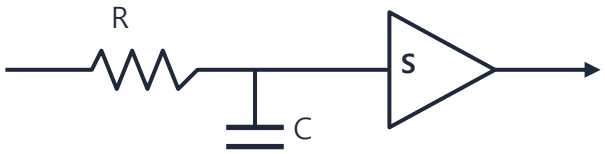
원인 FPGA = 10 ns마다 i_btn sampling · bounce pulse 폭 ≫ 10 ns → pulse 하나하나가 유효한 1로 인식

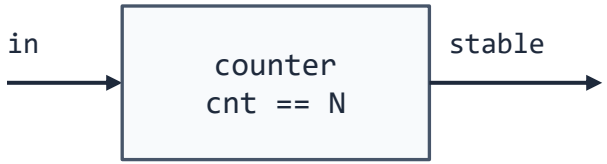
결과 i_btn의 bouncing에 따라 toggling 및 state 변화 / 입력 시점에 따른 Metastability 발생

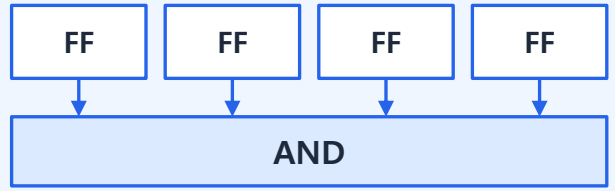
Three Ways to Debounce: RC Filter, Counter, Shift Register

01

	RC + Schmitt trigger 보드 회로	Counter 방식 RTL	Shift Register 방식 (SIPO) RTL · Homework 적용
원리	RC 시정수로 빠른 전압 변화 감쇠 + hysteresis 2단 문턱	입력 ≠ 출력 동안 count +1 · 입력 변화 시 count reset	최근 N sample을 shift register에 병렬 보관
판정 조건	**V > V_T+ → 1 · V < V_T- → 0**	**count == N → 출력 갱신**	**N bit 모두 1 → 1 (N-input AND)**
장점	FPGA logic 불필요	긴 window를 counter bit 수로 구현	판정 = AND 1단 · latency N cycle 고정
한계	부품 추가 · 시정수 고정 (변경 = 보드 수정)	comparator · reset 경로 필요	window = N × sampling 주기 → 긴 window엔 sampling enable 필요

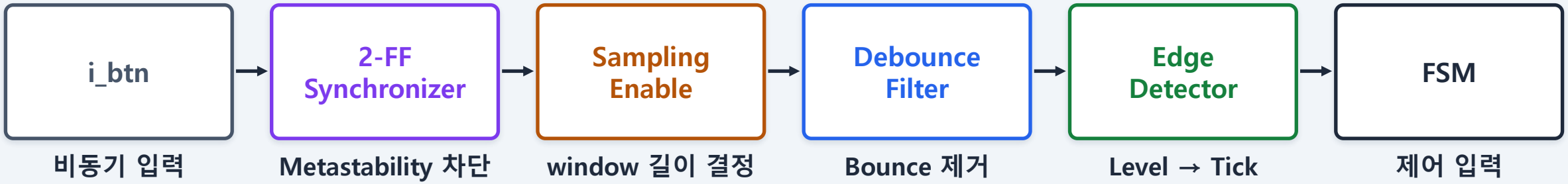






Production Debounce Chains Synchronizer, Filter and Edge Detector

01



button_debouncer 대응

2-FF Synchronizer	q_reg[7] 1단이 sampling 겸용 (전용 synchronizer 없음)
Sampling Enable	없음 → 100 MHz 매 posedge sampling → window 80 ns
Debounce Filter	q_reg[7:0] 8-bit SIPO + 8-input AND (debounce)
Edge Detector	edge_reg + ~edge_reg & debounce → o_btn 1-cycle tick

*window – N(samplie number) x Sampling frequency

SIPO Checks the Last 8 Samples at Once

01

q_reg 8 sample → 8-input AND



1 Serial In

매 posedge i_btn 1 bit → q_reg[7] 진입, 나머지 right shift

2 Parallel Out

8 bit 동시 노출 → 8-input AND 1단으로 판정

3 판정 의미

최근 8 × 10 ns = 80 ns 구간 전체가 1인지 검사

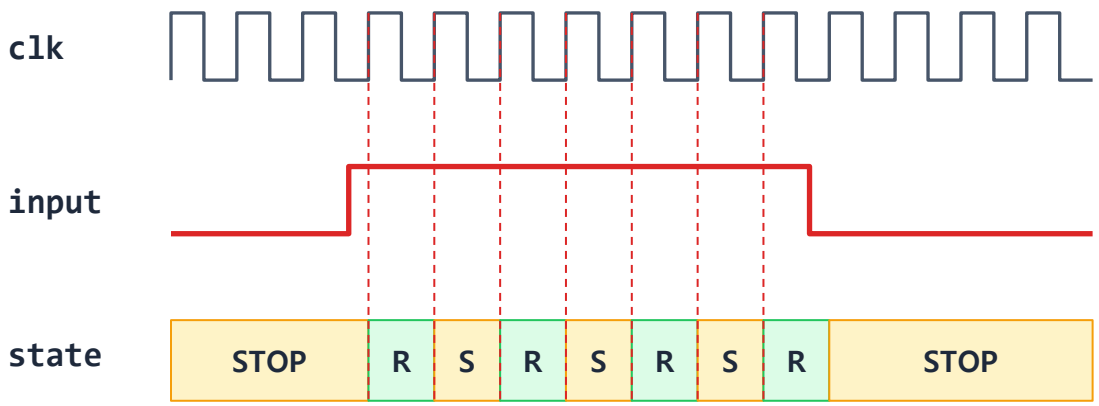
4 Counter 대비

comparator · reset 경로 불필요, latency 8 cycle 고정

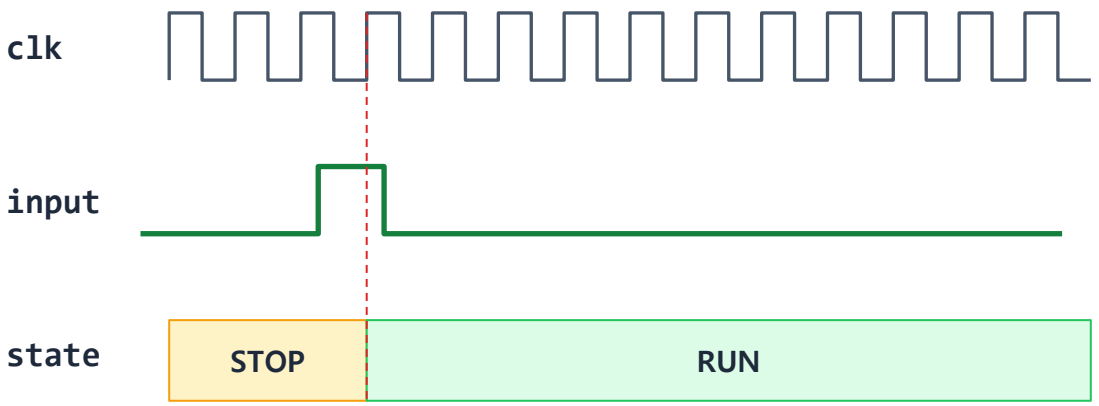
Level Input Toggles Every Clock, a Tick Toggles Once

01

Level 입력



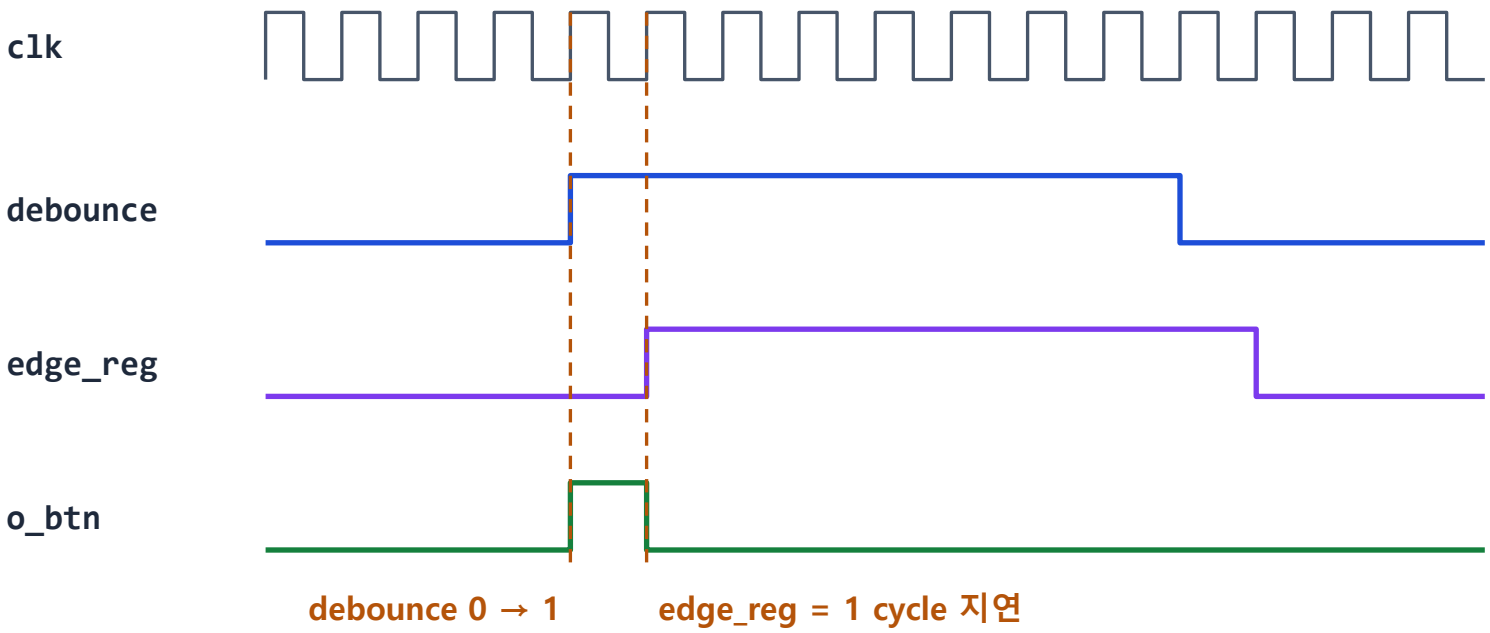
Tick 입력



o_btn = ~edge_reg & debounce Makes a One-cycle Tick

01

Edge Detect Timing



o_btn Truth Table

debounce	edge_reg	o_btn	의미
0	0	0	미입력
1	0	1	rising
1	1	0	유지
0	1	0	falling

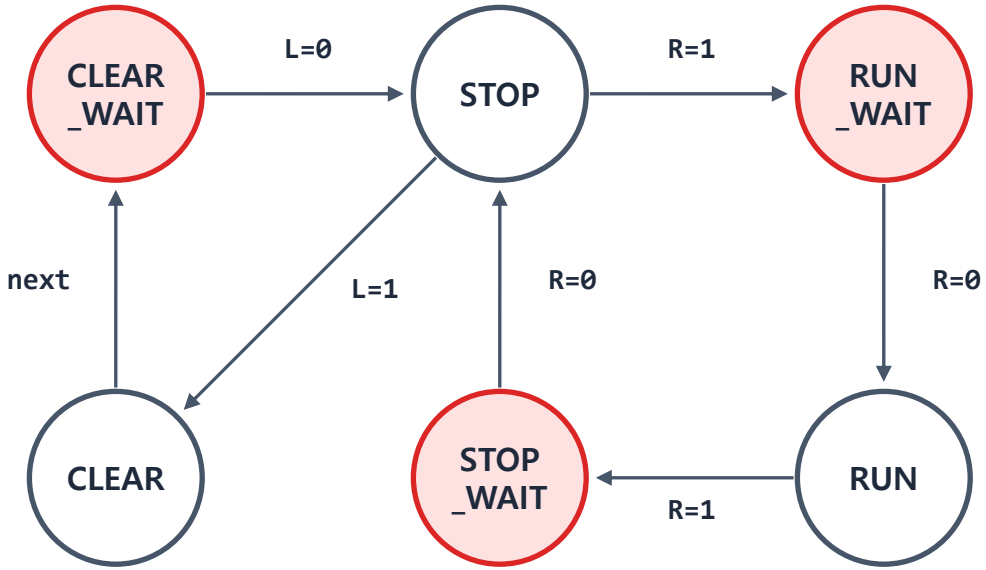
1. push 1회 = 이벤트 1회
2. push length 무관
3. clk 동기 1-cycle pulse

A Tick Keeps FSM_RUN_STOP at Three States

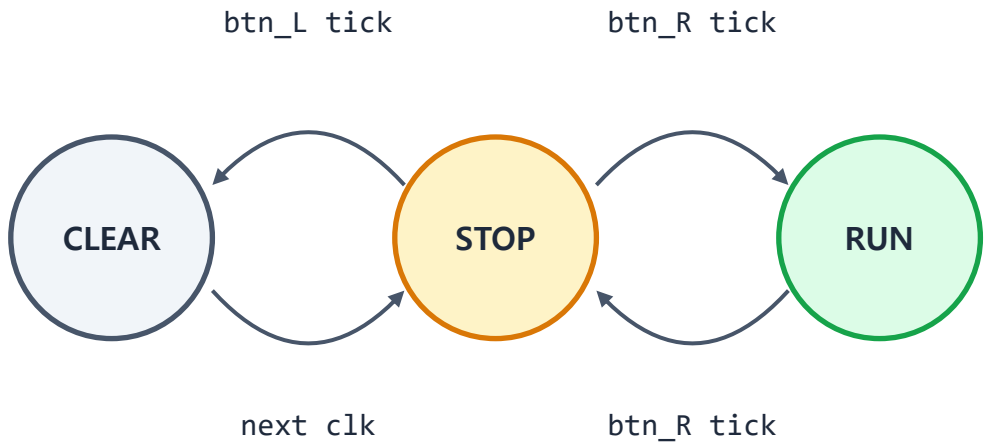
01

Level 입력 FSM

release 대기 state 필요



Tick 입력 FSM: 3 state



1 state 수 6 → 3

release 대기 state 제거

2 역할 분리

edge 검출 = debouncer, FSM = 순수 제어

3 단일 clock domain

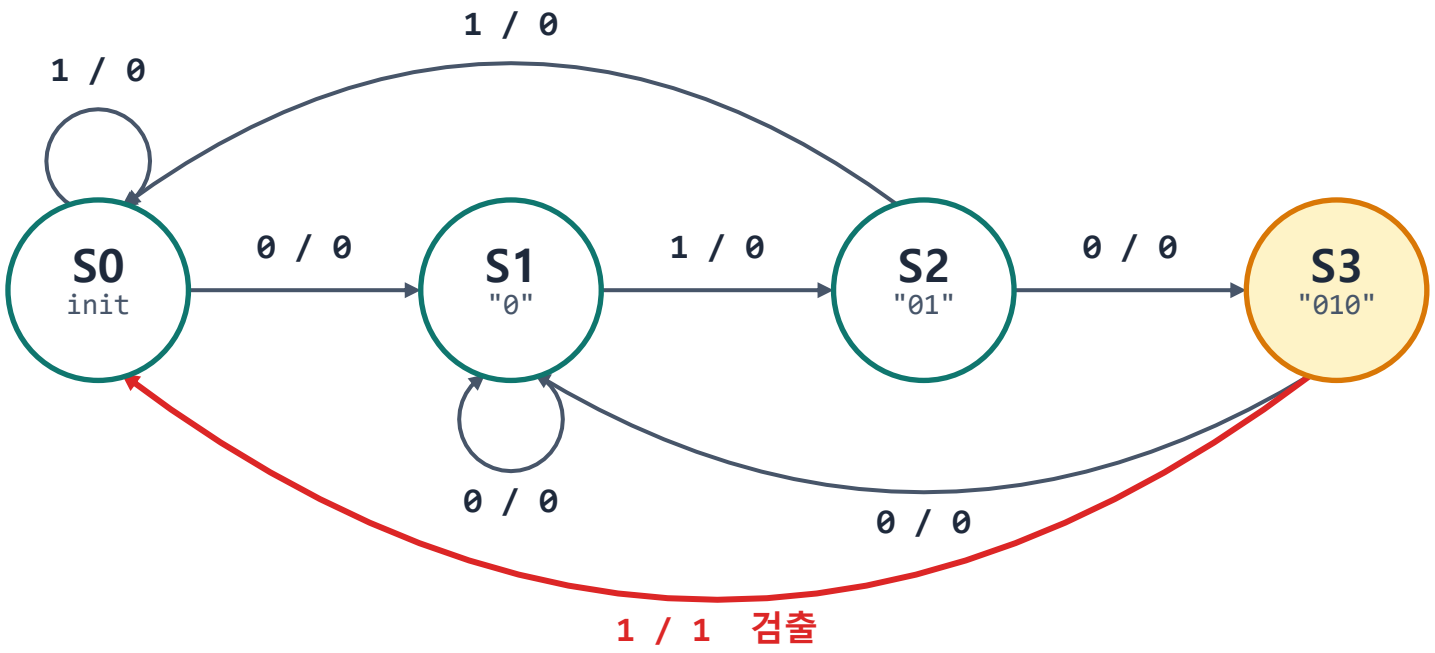
tick = enable 조건 → gated clock 불필요

4 재사용성

1-cycle pulse → counter · 다른 FSM에 그대로 연결

Mealy: 4 States Detect 0101, Output Depends on din_bit

edge label = din_bit / mealy_out



State 의미

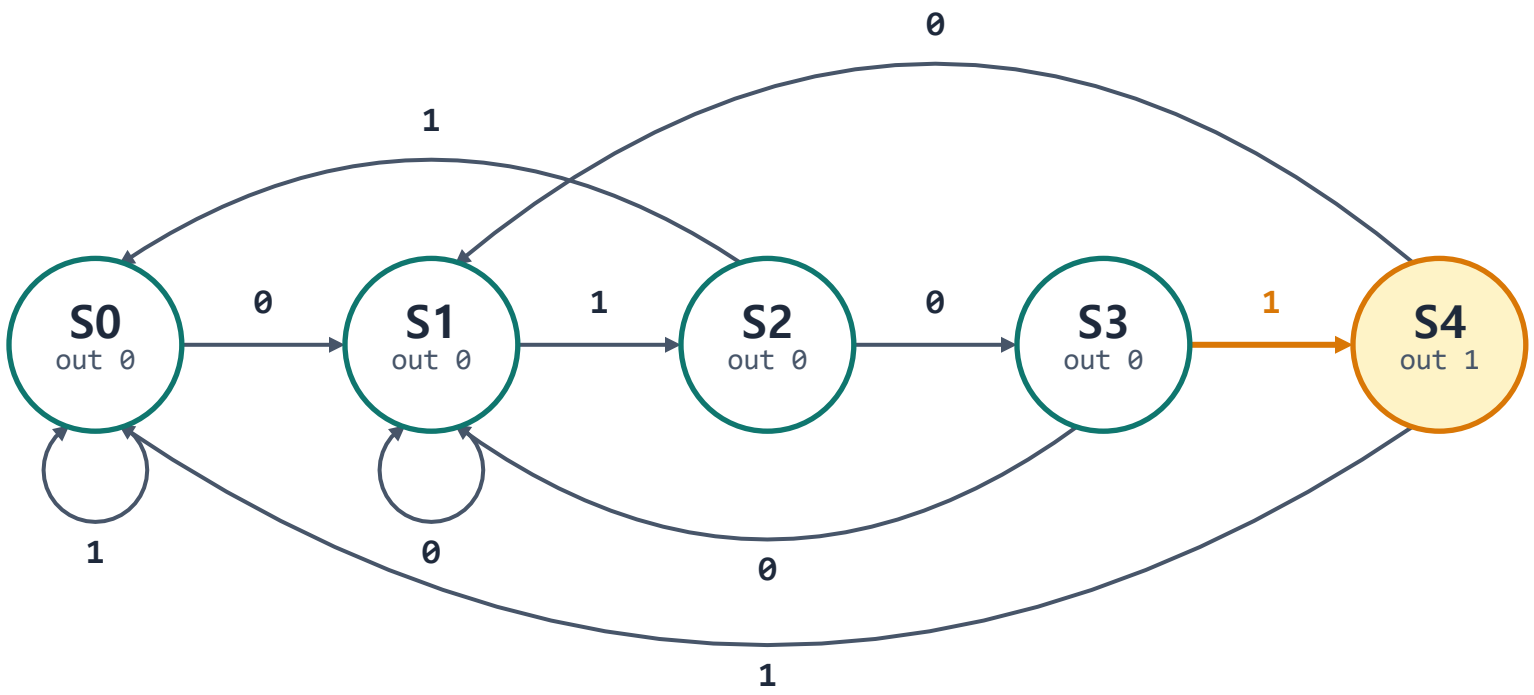
State	수신 이력	mealy_out
S0	없음 · 초기	0
S1	"0"	0
S2	"01"	0
S3	"010"	din_bit

mealy_out = (S3) · din_bit
→ 입력이 출력에 직접 연결

Non-overlap: 검출 후 S0 복귀 → 마지막 1 재사용 없음

Moore: S4 Holds the Detection, Output Depends on State Only

edge label = din_bit



State 의미

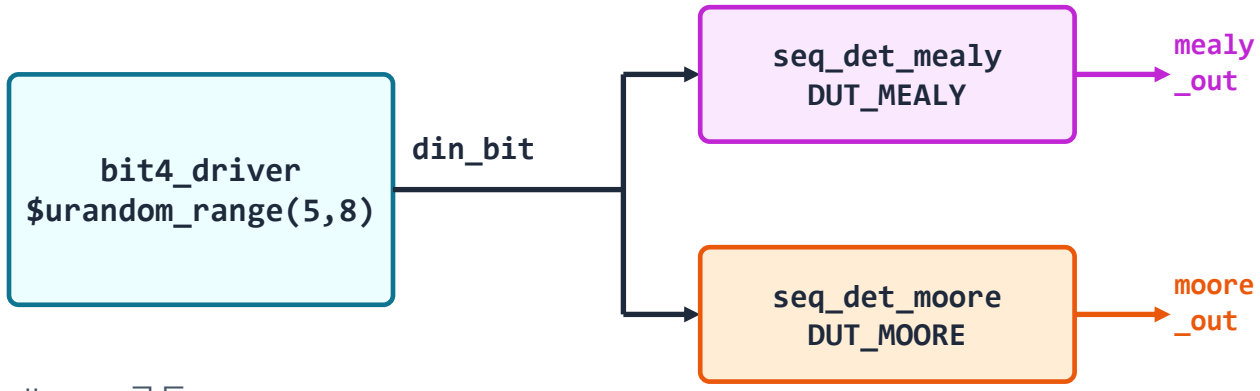
State	수신 이력	moore_out
S0	없음 · 초기	0
S1	"0"	0
S2	"01"	0
S3	"010"	0
S4	"0101"	1

moore_out = (state == S4)
→ state만으로 출력 결정

Non-overlap: S4에서 0 → S1, 1 → S0

Both Machines Receive the Same Serial din_bit

Testbench 구성 (tb_dut)



clk · rst 공통

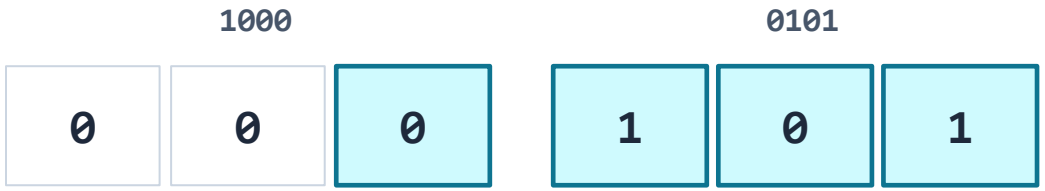
입력 조건

- **clk** : 10 ns 주기 (100 MHz)
- **rst** : 0 ~ 15 ns 동안 1 (async reset)
- **bit4** : 5 ~ 8 random, 50회 반복
- **인가 순서** : bit4[0] → [1] → [2] (LSB부터 3 bit)
- **변경 시점** : negedge + 1 ns

bit4 → din_bit serial

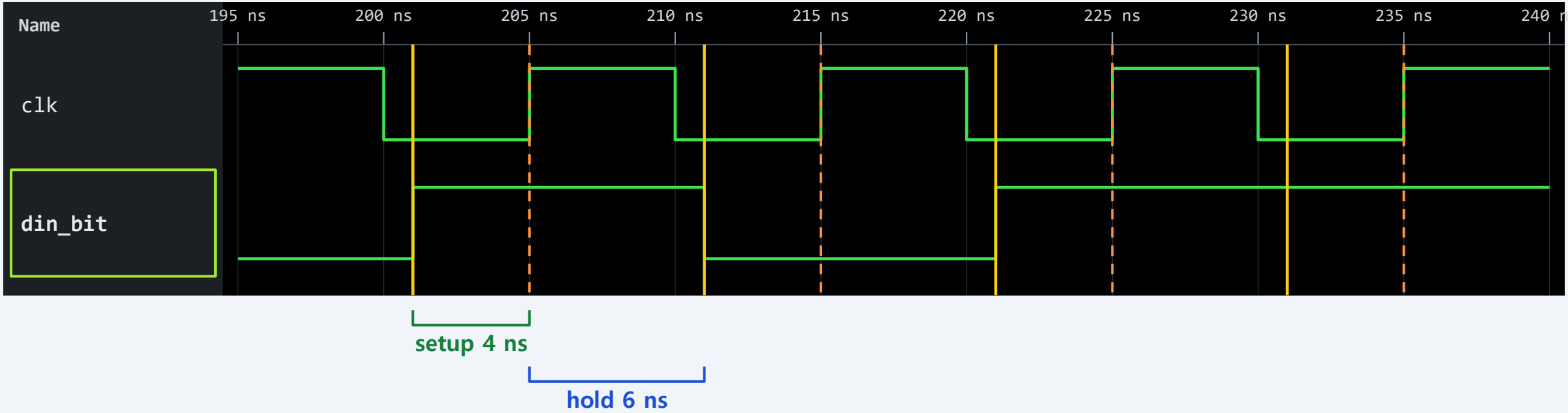
0101 (5)	1 → 0 → 1	0110 (6)	0 → 1 → 1
0111 (7)	1 → 1 → 1	1000 (8)	0 → 0 → 0

0101 형성 예 (161 ~ 225 ns)



단어 경계를 넘어 "0101" 완성 → 221 ns 검출

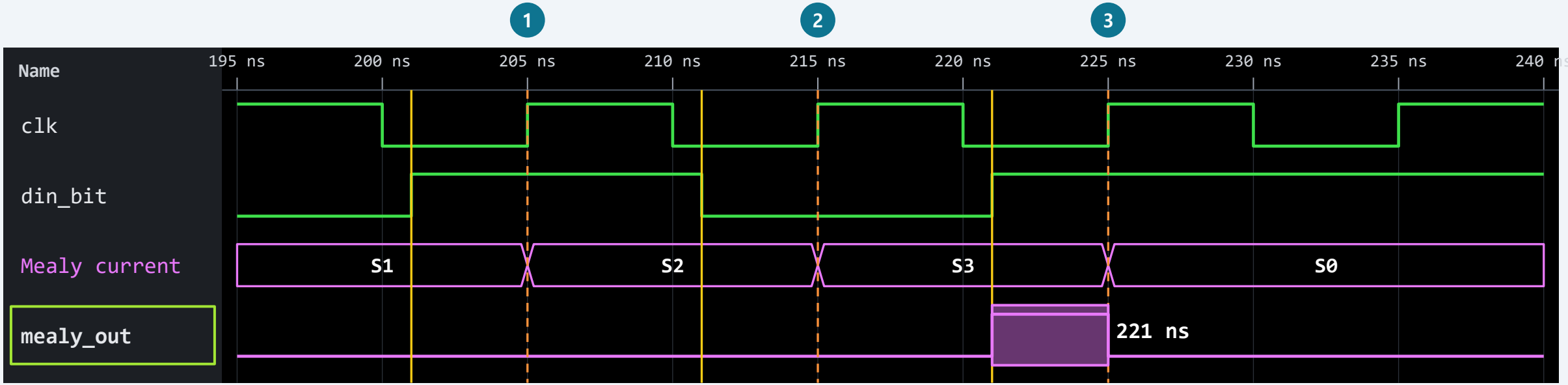
din_bit Changes at negedge + 1 ns, Sampled at posedge



1. **변경:** negedge + 1 ns (노란선)
2. **Sampling:** posedge (주황 점선)
3. **Margin:** setup 4 ns · hold 6 ns

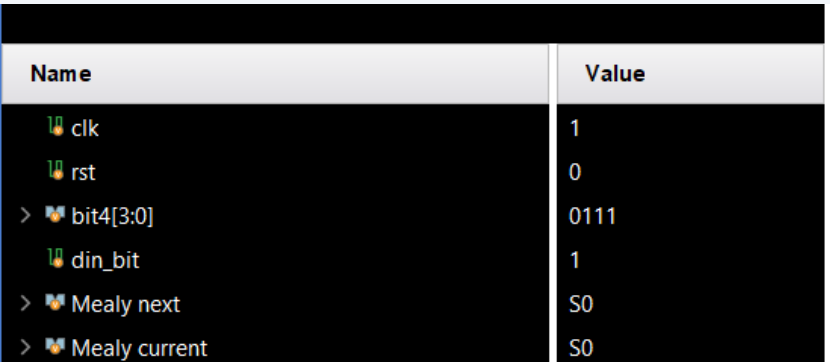
posedge 전후 모두 여유 확보 → 두 machine이 같은 안정된 값을 sampling

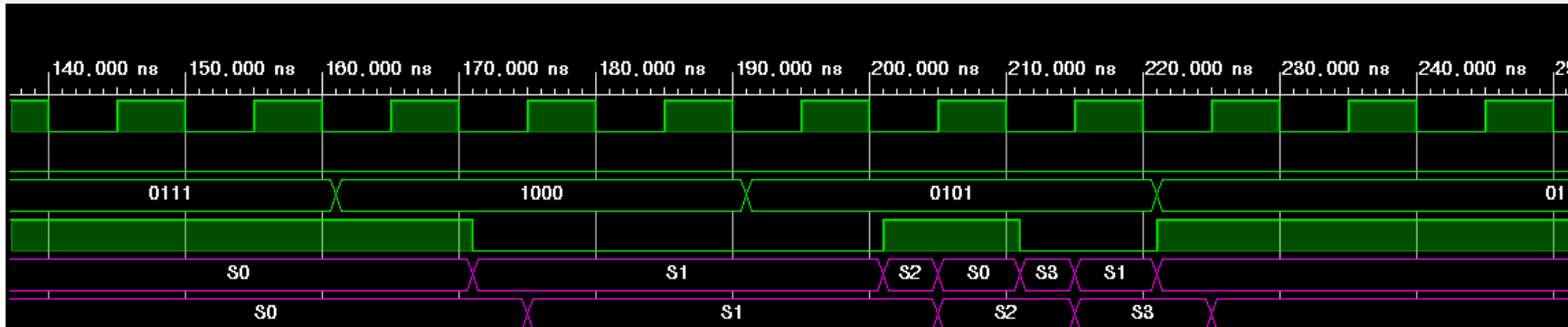
Mealy: Output Rises the Moment the Last 1 Arrives



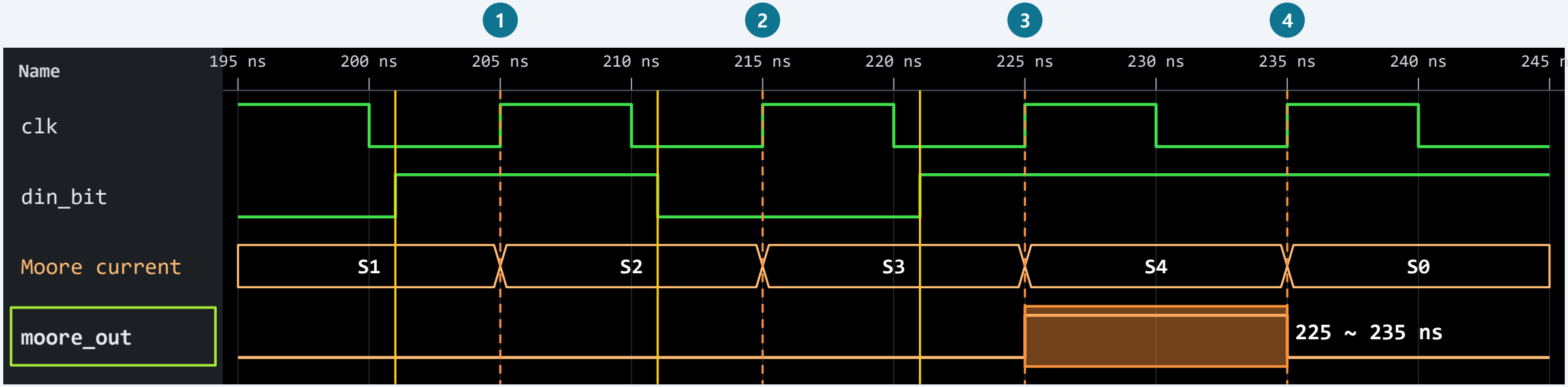
① din 1 ② din 0 ③ din 1







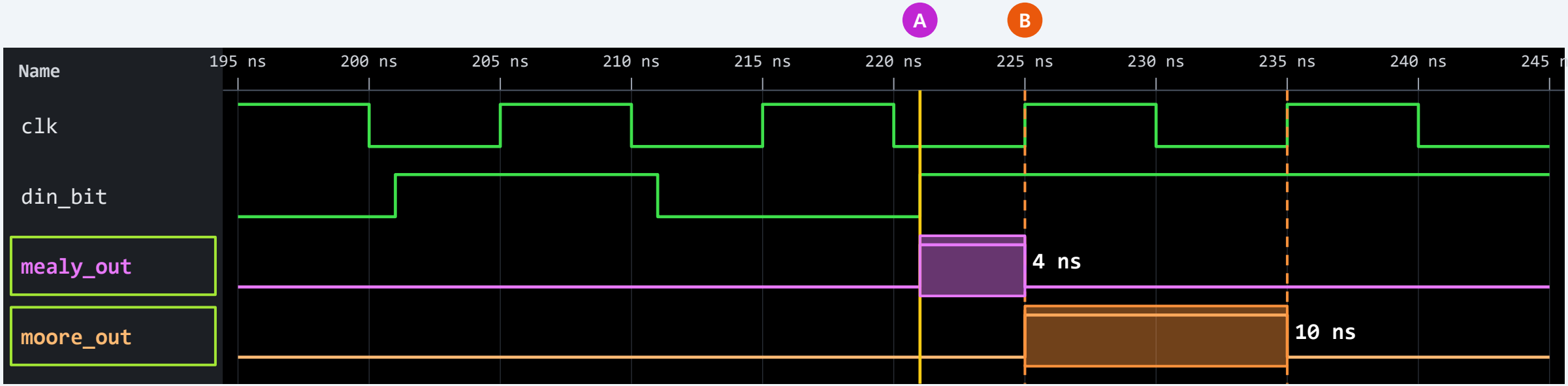
Moore: Output Rises One Clock Later in S4





moore_out = 1 @ 225 ns : posedge 225 ns에 S4 진입 후 1 clock 유지
출력 = state 해독 → din_bit 도착(221 ns) 후 다음 posedge까지 대기

Mealy Asserts 4 ns Earlier, Moore Holds a Full Clock



A **Mealy: 221 ns, din_bit 도착 즉시**

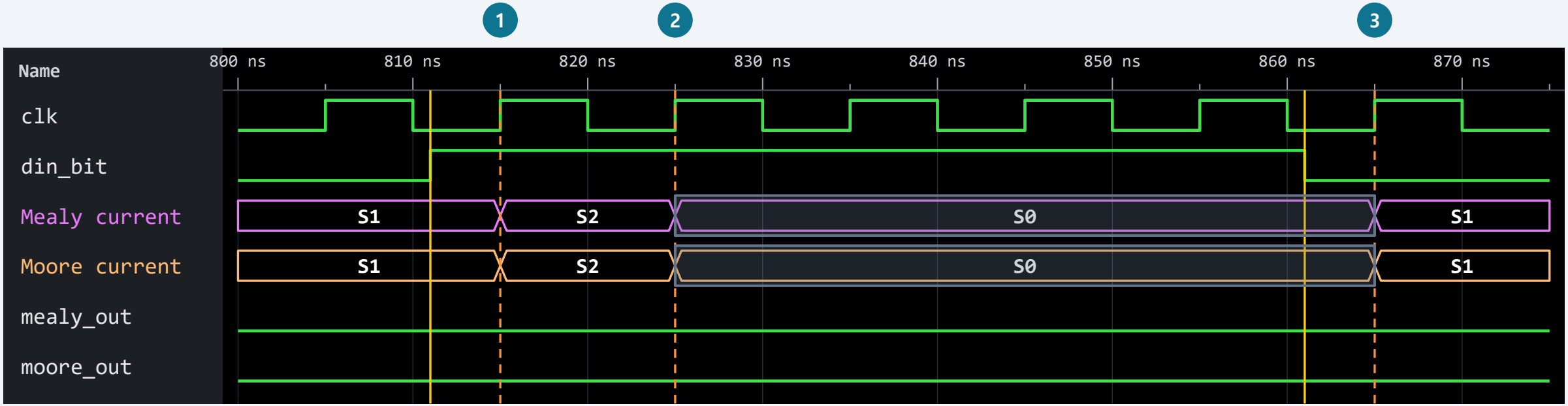
- 폭 4 ns = din_bit 변경 ~ 다음 posedge 구간
- posedge에서 state 이탈 → 출력 해제

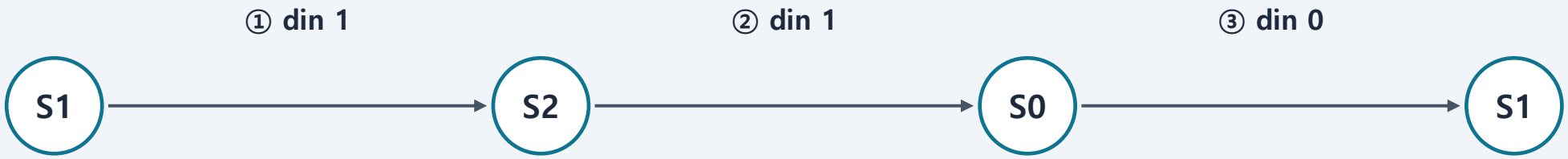
B **Moore: 225 ns, posedge 이후**

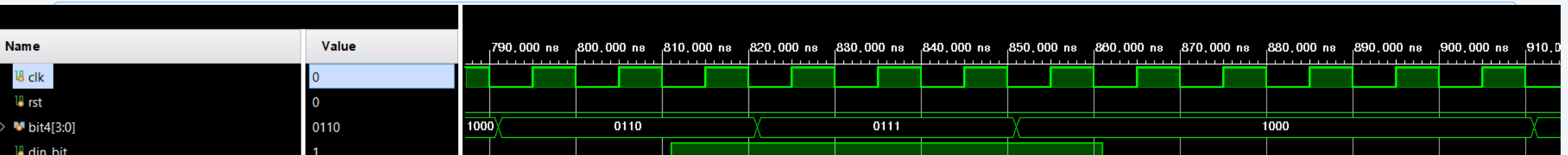
- 폭 10 ns = S4 체류 1 clock
- clock 경계에 정렬된 출력

Mealy = combinational 출력 (state + input) · Moore = register 기반 출력 (state)

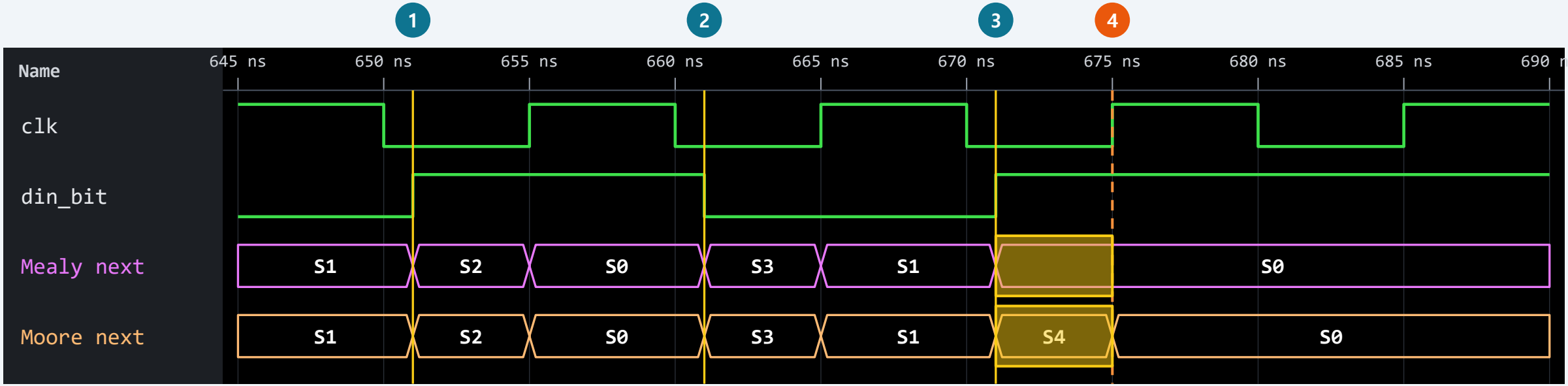
Without 0101, Both Machines Stay in S0–S2 with Output 0







Next-state Logic Differs Only at S3 with din_bit = 1



Mealy next = S0

검출 → 출력 logic이 담당 (mealy_out = 1), state는 S0 복귀

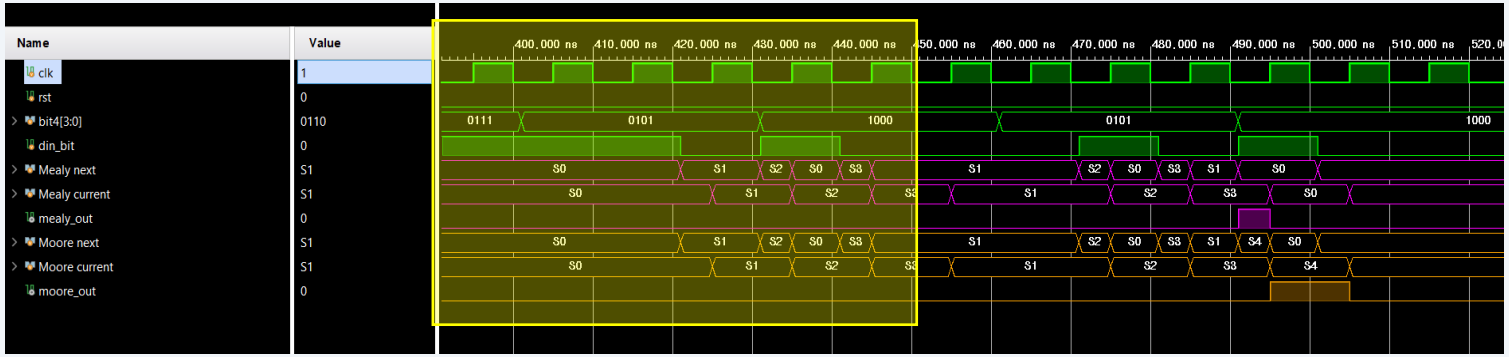
Moore next = S4

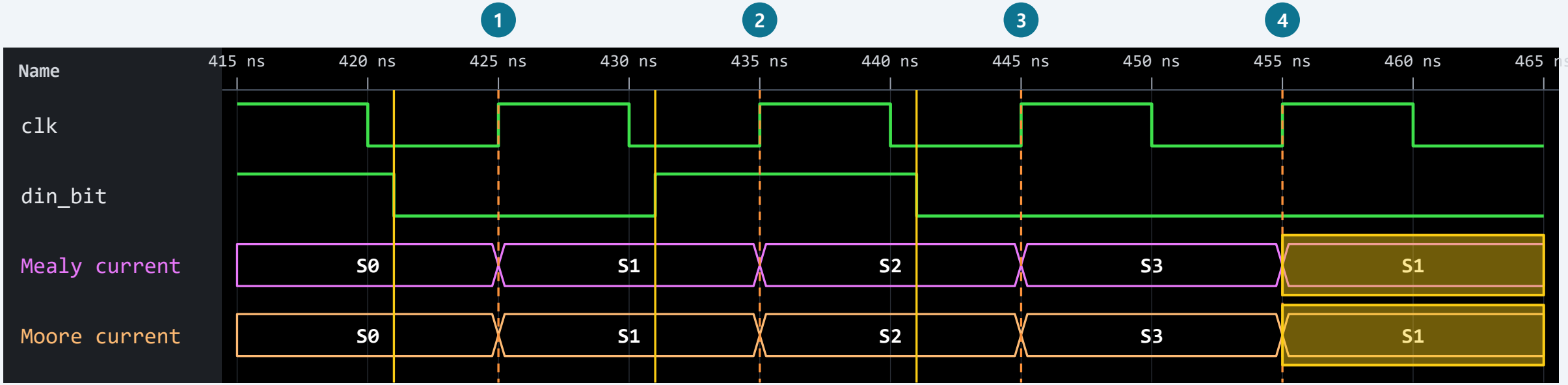
검출 → state로 저장 → S4 decode로 출력

①②③ din_bit 변경 즉시 next 값 변화 → next = 조합 logic
④ posedge 675 ns 순간의 next 값만 state로 반영

나머지 천이 전부 동일 → 차이 = S3 · din_bit 1 한 경우 (S0 vs S4)

S3 with din_bit = 0 Returns Both Machines to S1





① din 0 ② din 1 ③ din 0 ④ din 0



Mealy Responds Faster, Moore Gives a Clock-aligned Output

04

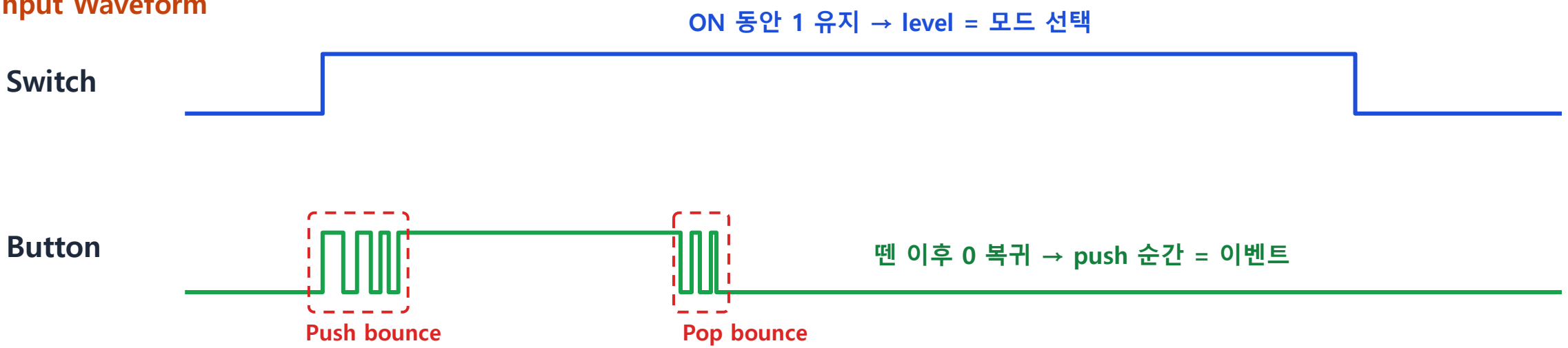
항목	Mealy	Moore
State 수	4	5 (S4 추가)
출력 결정	state + din_bit	state only
출력 시점	221 ns (din_bit 도착 즉시)	225 ns (posedge 이후)
출력 폭	4 ns (din_bit 변경 ~ posedge)	10 ns (1 clock)
Input glitch	출력에 그대로 전달	state FF에서 차단
적합한 용도	빠른 응답 필요 경로	다음 단 FF 입력 · 안정 출력

응답 속도 = Mealy 우위 (4 ns 빠름) · 출력 안정성 · timing 정렬 = Moore 우위

Buttons Return to 0 on Release, Unlike Switches

05

Input Waveform



1 FSM 입력 방식 변경

level → 1 cycle tick

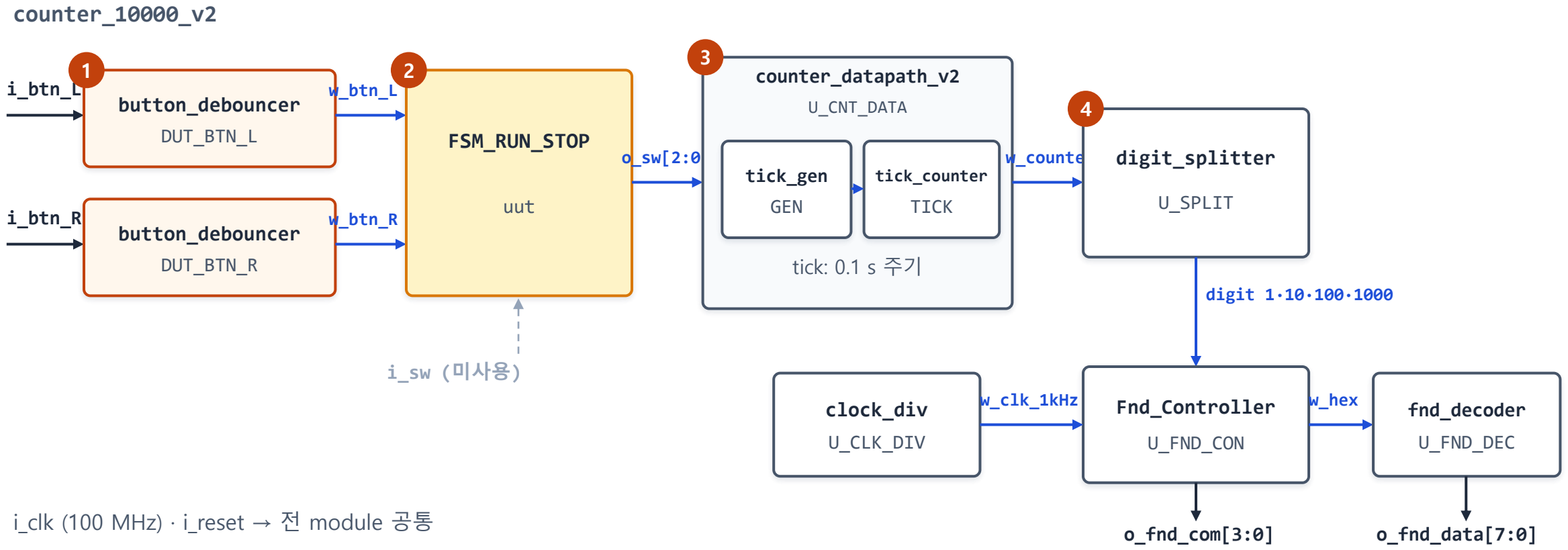
- 스위치 level 그대로 사용 시 push 유지 동안 매 clk 천이
- 천이 조건 = button_debouncer의 o_btn tick

2 Debouncer 추가

Push · Pop bounce 제거

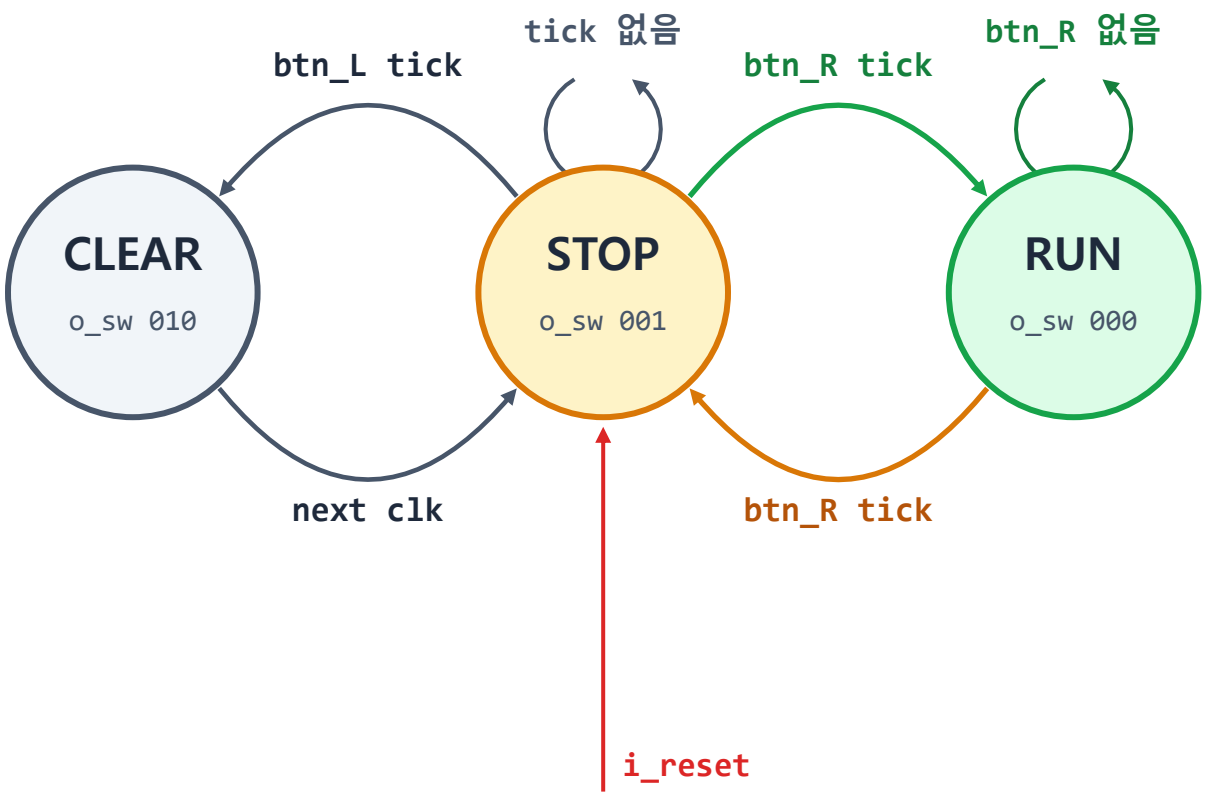
- bounce pulse마다 tick 발생 위험
- 8 sample 연속 1일 때만 유효 입력

counter_10000_v2: Debounced Buttons Drive FSM_RUN_STOP



1. **button_debouncer** bounce 제거 + 1 cycle tick
2. **FSM_RUN_STOP** tick → STOP · RUN · CLEAR
3. **counter_datapath_v2** o_sw에 따라 HOLD · CLEAR · count
4. **digit_splitter → FND** 자리 분리 → scan → segment

FSM_RUN_STOP: btn_R Toggles RUN / STOP, btn_L Clears



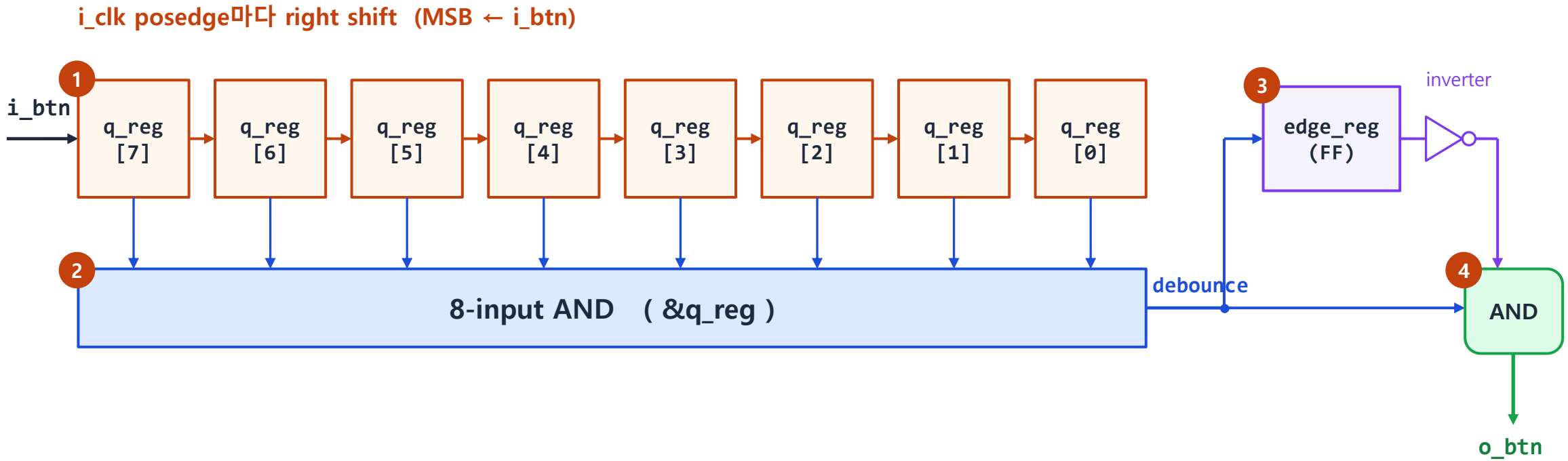
o_sw → tick_counter (i_tick 시)

State	o_sw[2:0]	동작
STOP	001	HOLD
CLEAR	010	0 로드
RUN	000	count +1

RUN 유지 조건 수정

- **수정 전** : RUN에서 btn_R 유무와 무관하게 STOP 천이 → RUN 1 clock만 유지
- **수정 후** : btn_R tick 없음 → RUN 유지, btn_R tick → STOP

button_debouncer: Sample, Check, Delay, Edge-detect



1 Sample
q_reg 8 bit 저장

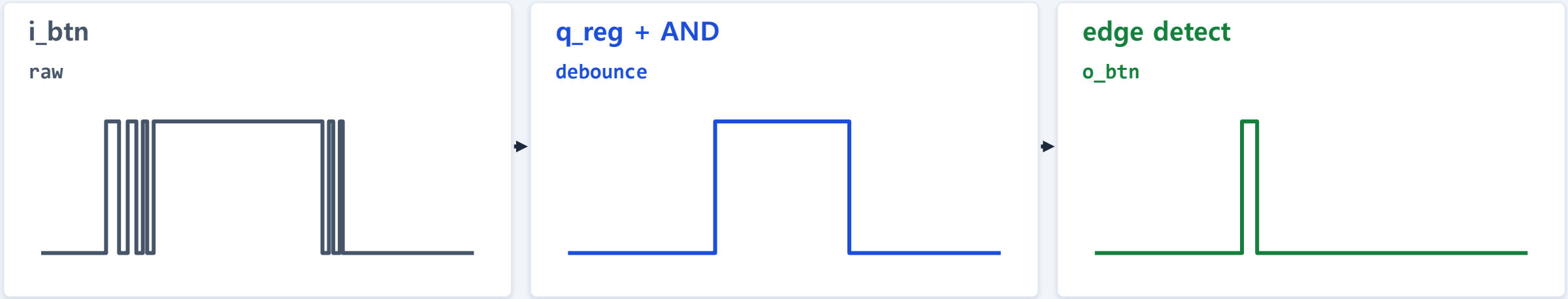
2 Check
8 bit 모두 1 판정

3 Delay
debounce 1 cycle 지연

4 Edge-detect
0 → 1 순간만 출력

Each Stage Removes One Failure Mode of a Raw Button

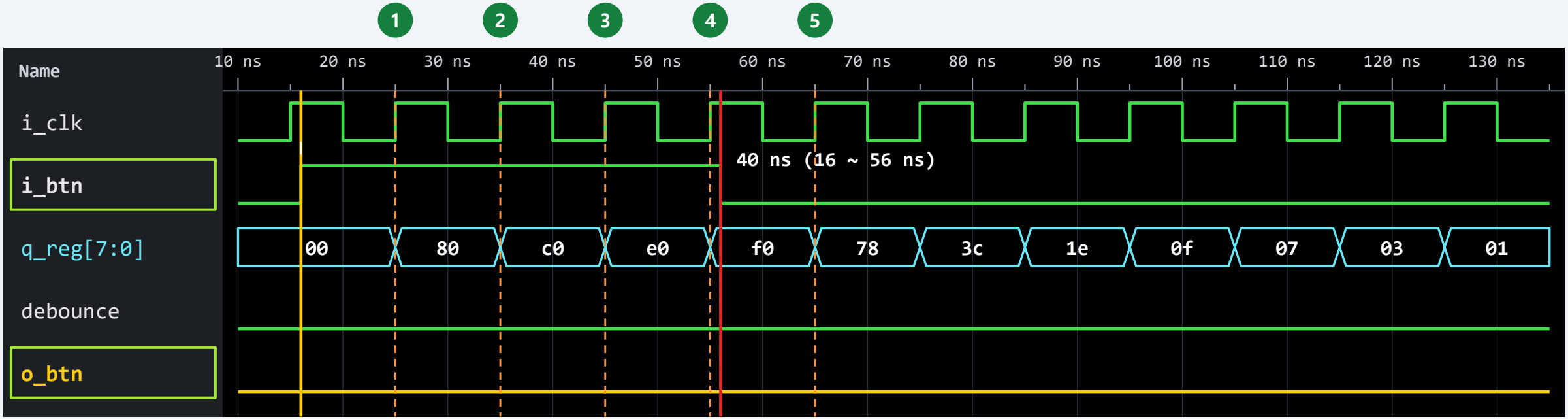
05

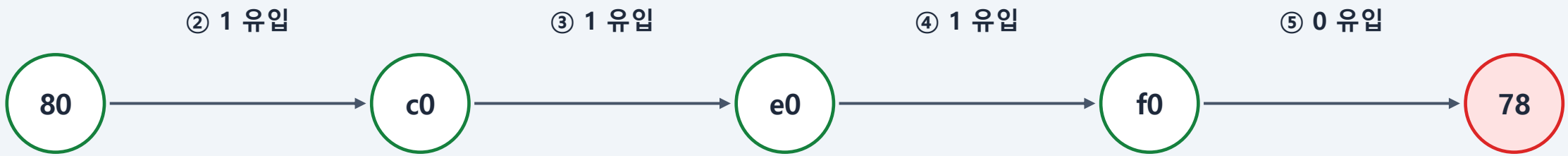


구성	역할	제거 대상
q_reg 8-bit SIPO	매 posedge i_btn sampling → clk 동기 · 최근 80 ns 이력 보관	**비동기 edge**
8-input AND	8 sample 모두 1일 때만 1 → 80 ns 미만 pulse 차단	**Bounce (scattering noise)**
edge_reg	debounce를 1 cycle 지연 → 직전 cycle 값 보관	**—**
~edge_reg & debounce	직전 0 · 현재 1인 cycle만 1 → push 1회 = tick 1회	**push 유지 중 반복 toggle**

A 40 ns Pulse Never Fills q_reg, So No Tick

06

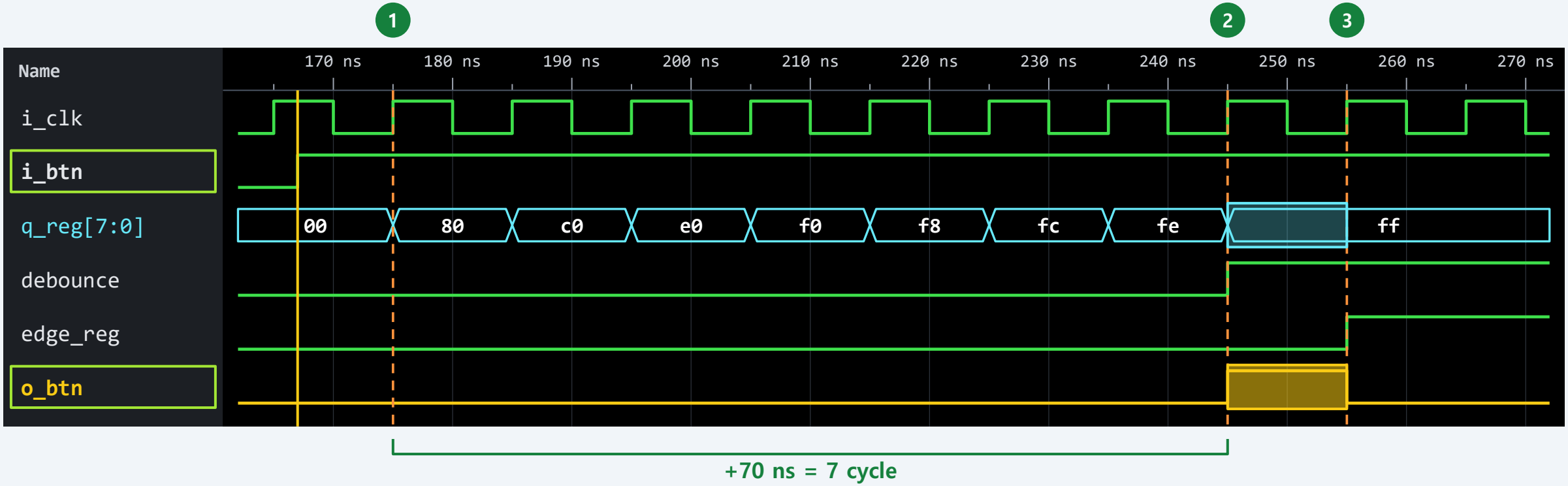




i_btn 40 ns → 1 sample 4개 (① ~ ④) → ff 미도달 → AND = 0 → o_btn 없음

o_btn Rises 70 ns After the First Sample and Lasts One Clock

06



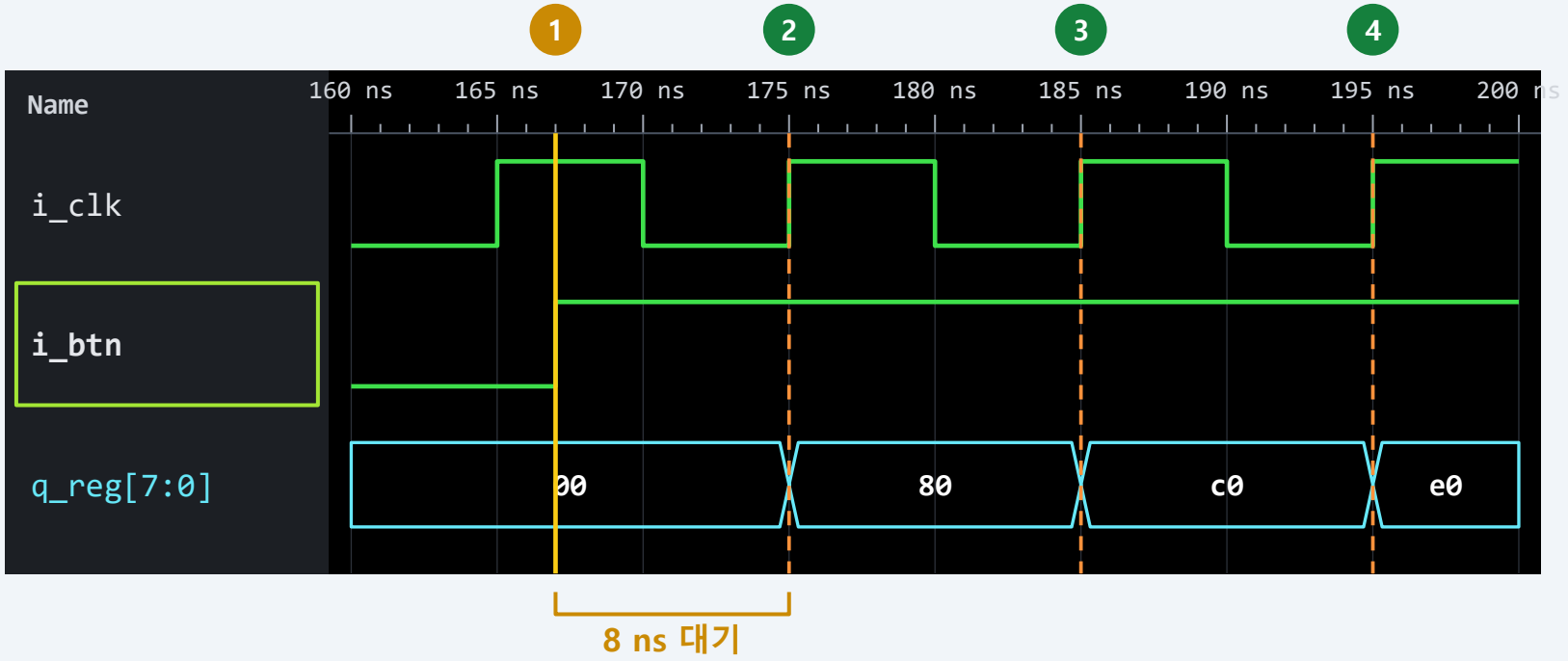
1 **175 ns**
첫 sample → q_reg = 80

2 **245 ns (+70 ns)**
8번째 sample → ff → debounce · o_btn = 1

3 **255 ns (+80 ns)**
edge_reg = 1 → o_btn = 0 (1 cycle)

debounce와 edge_reg의 and 조합으로 edge 검출(debounce의 상승 edge)

An Off-grid Edge Is Captured at the Next posedge



TB driver (async 재현)

- **i** = $urandom_range(5, 12) ns
- **j** = $urandom_range(5, 15)
- **repeat(j) #(i)** → 첫 i 후 i_btn = 1
- **High 폭** = (j − 1)·i + 10 ns
- **Low 폭** = 100 ns + 다음 i

1 **167 ns**
i_btn 0 → 1 (grid 밖)

2 **175 ns**
q_reg = 80

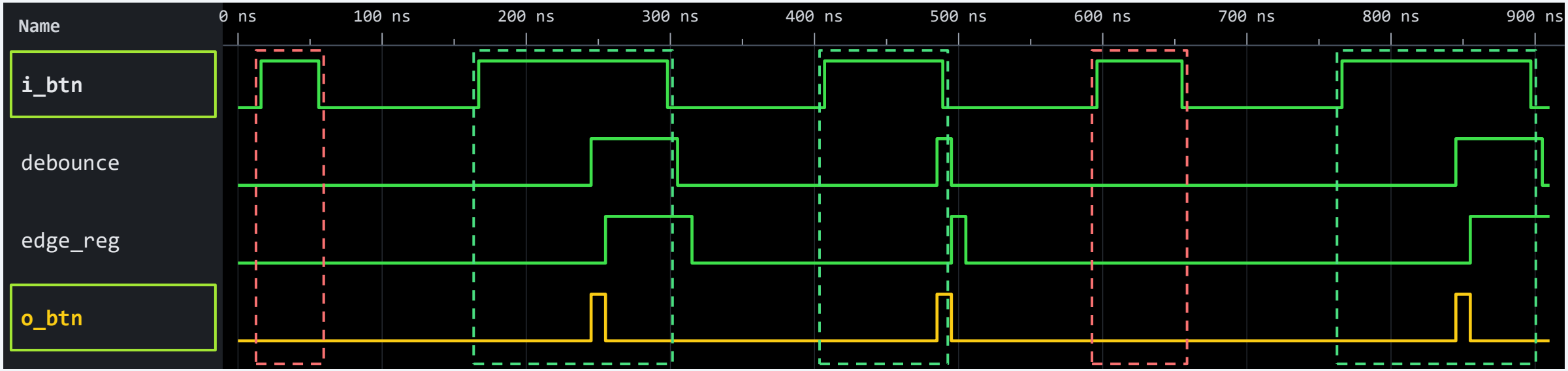
3 **185 ns**
q_reg = c0

4 **195 ns**
q_reg = e0

두 posedge 사이 변화 → 다음 posedge에서 q_reg[7]로 sampling · MSB부터 1 누적

Pulses of 40 ns and 59 ns Are Rejected, 82 ns and Longer Pass

06



40 ns ✗ 131 ns ✓ 82 ns ✓ 59 ns ✗ 131 ns ✓

o_btn 없음 : 40 ns · 59 ns
→ sampled 1이 4개 · 5개

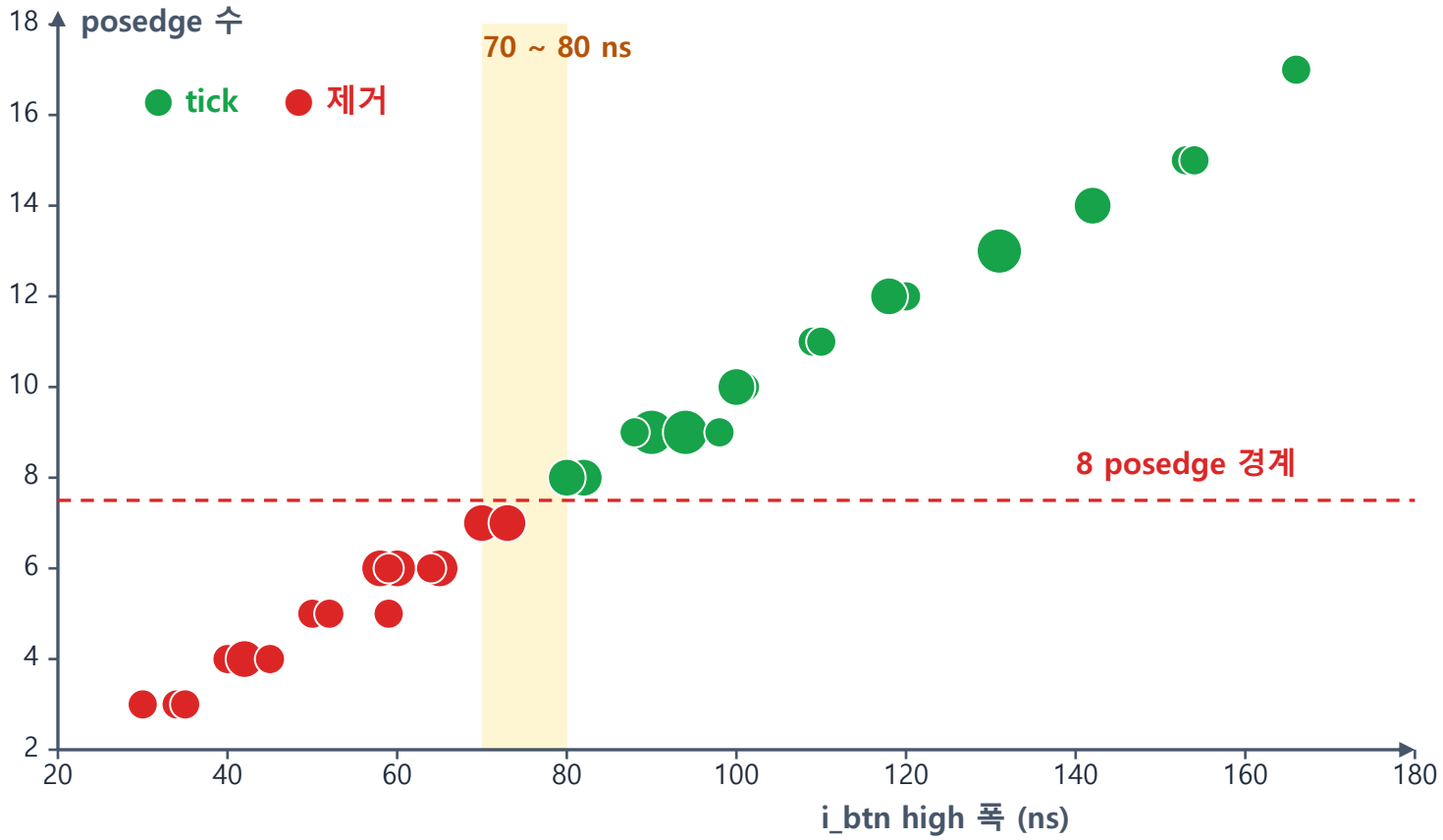
o_btn 1 tick : 82 ns · 131 ns
→ sampled 1이 8개 · 13개

tick 위치 245 · 485 · 845 ns = 각 pulse의 첫 sample + 70 ns · pulse length와 무관하게 1 tick

50 Random Presses: 8 Sampled Edges Always Tick, 7 Never Do

06

Pulse 폭 vs sampled posedge 수 (tb_button_debouncer, 50회)



28 tick 발생

8 ~ 17 posedge · 80 ns 이상 전부

22 제거

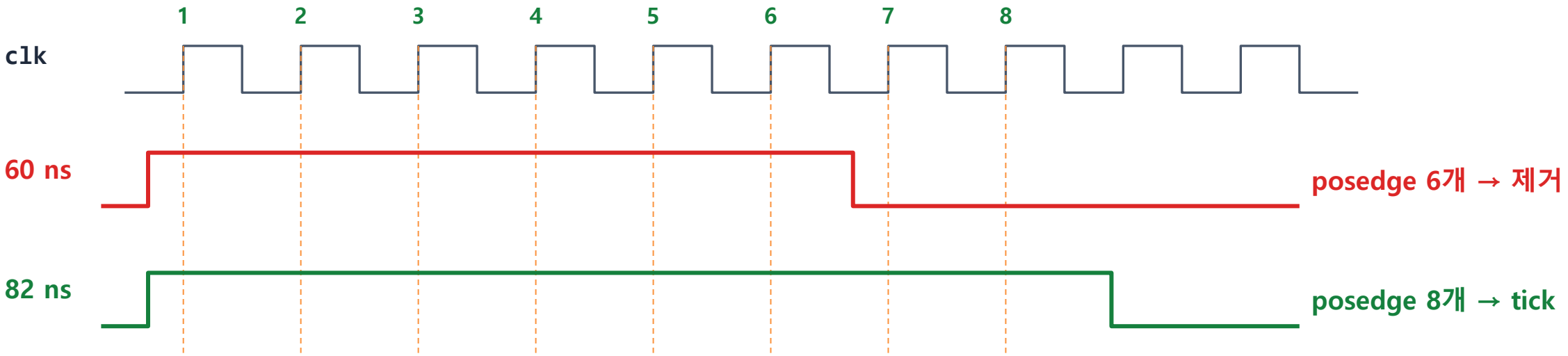
3 ~ 7 posedge · 73 ns 이하 전부

70 ~ 80 ns 구간

70 · 73 ns 4건 = 7 posedge → 제거
판정 기준 = sampled posedge 수

A Tick Needs i_btn High Across 8 Rising Edges

06



첫 sampling edge 이후 70 ns + α 유지 = 8번째 posedge까지 1

70 ns 이하

최대 7 posedge
항상 제거

70 ~ 80 ns

7 또는 8 posedge
clk edge 정렬에 의존

80 ns 이상

8 posedge 이상
항상 통과

Conclusion

07

구분	결과
Homework-1	Mealy: 4 state · 출력 = S3 · din_bit → din_bit 도착 즉시 검출 (221 ns, 폭 4 ns)
Homework-1	Moore: 5 state · S4 decode 출력 → 1 clock 뒤 검출 (225 ns, 폭 10 ns)
Homework-1	next logic 차이 = S3 · din_bit = 1 한 경우뿐 (Mealy S0 · Moore S4)
Homework-2	8 sample 연속 1 + rising edge → push 1회 = o_btn 1 tick (첫 sample + 70 ns)
Homework-2	50회 random push: 8 posedge 이상 28건 tick · 7 이하 22건 제거 · RUN 유지 조건 누락 수정
고찰	Button = event 입력 → tick 변환이 FSM 입력의 전제
고찰	8-sample window = moving average low-pass + 문턱 1.0 → 80 ns 미만 pulse 제거
고찰	80 ns window → ms 단위 bounce 대응 시 sampling 주기 확대 필요